

NRG Question Generation Pipeline

How to run it in three modes: provider-agnostic, Kimi, and Claude (Sonnet / Opus)

Script: scripts/generate-questions.ts · Rules: scripts/generation-rules.json (MCQ standard v0.1) · October 2026

1. What the pipeline does

The script turns a short config into RENR-style multiple-choice questions and refuses anything that breaks the MCQ authoring standard. The model only writes. The script plans, checks and stores. That split is what makes the three modes interchangeable: whichever AI writes the items, the same validator decides whether they are kept.

1. **Plan.** The script works out exactly how many items go to each RENR domain, taxonomy level and topic cluster, and numbers them as *specs*.
2. **Write.** A model writes one item per spec as JSON: stem, four options, the key, a type and a rationale for every option.
3. **Validate.** Every item is checked against the rules (section 7). Failures are retried (API modes) or listed with the reason (subscription modes).
4. **Review sheet.** Kept items are saved as JSON plus a readable .md sheet with each option's type shown.
5. **Insert (optional).** `--insert --env staging` writes them to staging as `is_ai_generated=true, is_active=false`. Nothing reaches students until a teacher approves it in `/teacher/review`.

2. Choosing a mode

	A. Provider-agnostic	B. Kimi	C. Claude (Sonnet / Opus)
Who writes the items	Any API: OpenAI, Gemini, Groq, OpenRouter, a local Ollama model, and others	Kimi K3 in Kimi Code CLI, on your Allegretto plan	Claude Code on your Max plan; Sonnet for drafting, Opus for hard items and review
Cost	Pay per token on that provider	Covered by Allegretto (5-hour and weekly limits)	Covered by Max (session and weekly limits)
Needs an API key	Yes	No (optional API fallback)	No (optional API fallback)
How it runs	Fully scripted: one command	Agent loop: brief, write, validate, fix	Agent loop with subagents in parallel
Best for	Large unattended batches when someone else's API is cheapest	Default bulk drafting on a flat-rate plan	Quality-sensitive batches and second-pass review of Kimi output

Recommendation: use B or C first, since you already pay for both plans. Use C for a batch that matters (Analysis-level items, anything going to Jade soon) and B for volume. Use A only when a batch is too big for the plan limits, or to try another model.

3. Setup common to every mode

3.1 One-time

- Node 20+ in the repo folder (`npx tsx` runs the script; nothing to build).
- `.env.local` already has the staging Supabase keys the script uses for `--dedup-db` and `--insert`. Add an AI key only for the API routes (section 4).
- Copy `docs/phase-1/generation-config.example.json` to a new file per batch and edit `title`, `total`, `audience` and `topic_emphasis`.

3.2 Always start with the plan

```
npx tsx scripts/generate-questions.ts --config gen.json --plan-only
```

This prints the domain / taxonomy / cluster counts and calls nothing. If the mix looks wrong, change the weights in the config before spending any model time.

3.3 Safety rules (all modes)

- Insert into **staging** only. Prod needs `--env prod` and `CONFIRM_PROD=yes`, and is Ian's decision, not an agent's.
- Never edit `generation-rules.json` or the script to make a failing item pass. Fix the item.
- Generated files go in `data/generated/`, which git ignores. Questions are content, not code.

4. Mode A: provider-agnostic (API)

One command does everything: plan, call the model in batches of 5, validate, retry failures up to 3 times, save. Choose the provider with `--provider`, and put the key in `.env.local`.

--provider	Key / env vars	Default model
openai	OPENAI_API_KEY	gpt-4o
gemini	GEMINI_API_KEY	gemini-2.0-flash
compatible	AI_BASE_URL, AI_API_KEY, AI_MODEL (any OpenAI-compatible endpoint: Groq, Together, OpenRouter, Mistral, DeepSeek, LM Studio)	AI_MODEL
ollama	OLLAMA_HOST (default <code>http://localhost:11434</code>), OLLAMA_MODEL	none: --model required
kimi / anthropic	See modes B and C (API fallback)	
mock	None: fake items, no network. Use it to check the setup.	

```
# 0. check the setup without spending anything
npx tsx scripts/generate-questions.ts --config gen.json --provider mock

# 1. small trial batch, saved locally, checked against what is already on staging
npx tsx scripts/generate-questions.ts --config gen.json \
  --provider openai --model gpt-4o-mini --total 10 --dedup-db --env staging

# 2. full batch, then insert into staging as pending review
npx tsx scripts/generate-questions.ts --config gen.json \
  --provider compatible --insert --env staging
```

Useful knobs: `--batch 3` if the model truncates long JSON; `--max-calls 40` to cap spend; `--temperature 0.5` for less variety; `--no-json-mode` if a compatible endpoint rejects `response_format`.

5. Mode B: Kimi (Allegretto plan)

5.1 Subscription route (default): Kimi Code CLI

Kimi Code writes the items in the terminal under your Allegretto membership, and the script only validates them. Allegretto unlocks the 1M-token context, so one session can hold the whole brief.

One-time: install Kimi Code, open it in the repo and sign in with your Kimi account.

```
curl -fsSL https://code.kimi.com/kimi-code/install.sh | bash
cd ~/Desktop/NRG && kimi
/login                # choose Kimi Code OAuth (your membership), not an API key
```

Each batch:

1. Write the brief (no model call): `npx tsx scripts/generate-questions.ts --config gen.json --prompt-out data/generated/setA.brief.md`
2. In Kimi Code, paste the agent prompt from section 8, with `setA` and `gen.json` filled in.
3. Kimi writes `data/generated/setA.items.json`, runs the validator, rewrites only the failed specs and repeats until 0 failures (at most 3 rounds).
4. You read `data/generated/setA.md` (the review sheet) and decide whether to insert.

Keep the config identical between the `--prompt-out` run and the `validate` run. Spec numbers come from the plan, so a different `total` or different weights would match items to the wrong specs.

Size: about 25 to 50 items per Kimi session works well. For 200, write the brief once and ask Kimi to fill it batch by batch (the brief is already split into batches of 5), validating after every 25. Allegretto's 5-hour window covers a few hundred requests; if Kimi reports a limit, stop and resume later. The items file keeps what's done.

5.2 API fallback

```
# .env.local: MOONSHOT_API_KEY=... (pay-as-you-go key from platform.kimi.ai)
npx tsx scripts/generate-questions.ts --config gen.json \
  --provider kimi --model kimi-k3 --effort high --dedup-db --env staging
```

The script sends no temperature to Kimi unless you pass `--temperature`, because some Kimi models accept only one fixed value. `--effort` maps to Kimi's `reasoning_effort` (low / high / max). This route is tested against a local fake server, not yet against Moonshot itself; run `--total 5` first.

6. Mode C: Claude, Sonnet or Opus (Max plan)

6.1 Subscription route (default): Claude Code

This works the same way as mode B, with one advantage: Claude Code can split the brief across **subagents** that write in parallel, then check and fix their output as one batch. Choose the model by job:

Job	Model	Why
Drafting most items (Knowledge, Application)	Sonnet	Fast, uses less of the Max allowance, and good enough when the validator is the gate
Analysis / Synthesis items, leadership and priority scenarios	Opus	Better at close-vs-priority distractors and multi-step clinical reasoning
Second-pass review of a finished batch (any mode)	Opus	Catches clinical errors and weak distractor types the regex rules cannot

Each batch:

1. Write the brief: `npx tsx scripts/generate-questions.ts --config gen.json --prompt-out data/generated/setB.brief.md`
2. Open Claude Code in the repo (Claude, or the VS Code extension). Pick the main model with `/model` (Opus for the coordinator is a good default).
3. Paste the agent prompt from section 8 and add the Claude line: *"Split the brief into groups of 10 specs and give each group to a Sonnet subagent; write Analysis-level specs (taxonomy ASE) with Opus."*
4. Claude merges the subagents' items into one `setB.items.json`, validates, fixes failed specs and reports.
5. Optional review pass, same session or a new one: *"Act as an RENR item reviewer. Read data/generated/setB.md and list any item whose key is clinically wrong, whose distractor types are mislabelled, or that tests above entry-level scope. Do not edit; just list."*

6.2 API fallback

```
# .env.local: ANTHROPIC_API_KEY=...
# (only for unattended runs; the Max plan does not include API credit)
npx tsx scripts/generate-questions.ts --config gen.json \
  --provider anthropic --model claude-sonnet-5-5 --effort high
npx tsx scripts/generate-questions.ts --config gen.json \
  --provider anthropic --model claude-opus-5-5 --effort high --batch 3
```

Current Claude models reject temperature, so the script no longer sends it. Depth is set with `--effort` (low / medium / high / xhigh / max; default high). Opus 5.5 defaults to *medium* if effort is not sent, so the script always sends it. A safety decline or a truncated reply is reported as a failed batch and retried.

7. What the validator rejects

These rules apply in every mode. In API modes a failing item is regenerated. In subscription modes the reason is printed so the agent can fix the item.

Rule	Rejected when
Shape	Not exactly 4 options labelled A to D; not exactly one key; blank option; "all/none of the above"; select-all-that-apply
Option types	option_types is not exactly one each of correct / close / priority / incorrect, or the key isn't typed correct
Rationales	Correct-answer rationale under 60 characters; any distractor rationale under 30
Scope	Critical-care / ICU terms, nurse practitioner / advanced-practice terms, the abbreviation NP (anywhere, rationales included)
Units	Blood glucose given in mmol/L. Caribbean practice uses mg/dL (e.g. 54 mg/dL, not 3.0 mmol/L)
Visual flagging	Bold or italic markdown, arrows or warning symbols, labels like (high), (low), (abnormal), (elevated) in the stem or options
Length cue	Option lengths spread wider than 1.4x, or the key is more than 12 characters longer than the distractor average
Duplicates	Stem identical to, or 70%+ word overlap with, one already in this batch or (with --dedup-db) in the database

Batch-level checks are printed as checks and saved in the JSON: the % of items where the key is the longest option (warns over 35%), the spread of correct-answer positions (warns if any letter is over 35%), and a count of rejections by rule.

Switches waiting on Jade (all off, in switches_pending_jade): block nursing-diagnosis items, drop the difficulty field, regional rather than Trinidad-and-Tobago wording, require abbreviations to be expanded, extra forbidden patterns. Flipping one changes both the prompt and the validator; no code change is needed.

8. The agent prompt (modes B and C)

Paste this into Kimi Code or Claude Code. Replace setA and gen.json.

```
You are writing RENR practice questions for the NRG platform.
```

1. Read data/generated/setA.brief.md. Its "System prompt" section is your authoring standard and JSON schema; follow it exactly. Its "Item specs" section lists every spec number with its domain, taxonomy and cluster.
2. Write ONE file, data/generated/setA.items.json, shaped {"items":[...]}, one item per spec, using the spec numbers given. Every item needs option_types (one each of correct/close/priority/incorrect) and a rationale for every option that cites data in the stem.
3. Validate (no model call, no database write):
npx tsx scripts/generate-questions.ts --config gen.json \ --provider file --input data/generated/setA.items.json \ --dedup-db --env staging --out data/generated/setA.json
4. Read "failures" in data/generated/setA.json. Rewrite ONLY those specs in setA.items.json, then run step 3 again. Stop at 0 failures or after 3 rounds.
5. Do not edit scripts/, generation-rules.json or the config to make items pass. Do not call any AI API. Do not use --insert.
6. Report: items kept / total, the "checks" line, any spec still failing and why, and any item you are clinically unsure of.

Then read data/generated/setA.md. If you're happy with it, insert into staging yourself (same config, same items file):

```
npx tsx scripts/generate-questions.ts --config gen.json \ --provider file --input data/generated/setA.items.json --insert --env staging
```

Option types in the database: question_options.distractor_type exists on staging only. The script writes it only when store_option_types is true in the rules file. Leave it false until the colour-coded rationale view is built; the types are still in the JSON and the review sheet.

9. Reading the output

File	What it is
setA.json	Everything: config, plan, kept items, checks, failures (with reasons) and warnings
setA.md	Review sheet: each item with its tags, the options labelled [CORRECT] / [CLOSE] / [PRIORITY] / [INCORRECT], and every rationale
setA.brief.md / setA.items.json	Subscription modes only: what the agent was given, and what it wrote

Warnings don't block an item. The usual ones are a model re-tagging a spec's domain or taxonomy (the planned tag is kept) and abbreviations that aren't expanded (logged only, until the abbreviation switch is on).

10. Troubleshooting

Symptom	Fix
Every item fails with the same rule	The model ignored that rule. Check the failures in the JSON. In API modes, try <code>--batch 3</code> or a stronger model.
"no JSON object in response"	The model wrapped the JSON in prose or ran out of room. Lower <code>--batch</code> ; for compatible endpoints try <code>--no-json-mode</code> .
"response hit max_tokens" (Claude API)	Lower <code>--batch</code> to 3.
"model declined" (Claude API)	A safety classifier stopped the batch. It is retried. If it repeats, check that the topic is phrased clinically.
Many "duplicate / near-duplicate stem"	The pool is saturated for that cluster. Change <code>topic_emphasis</code> or the cluster weights.
file mode: a spec says "no item returned"	The items file is missing that spec number, or the config changed since <code>--prompt-out</code> .
Missing SUPABASE_URL_STAGING	Run from the repo root so <code>.env.local</code> is picked up.
Kimi or Claude plan limit hit mid-batch	Stop. The items file keeps what's done. Resume in the next window and ask the agent to fill only the missing specs.

11. Flag reference

Flag	Meaning
<code>--config P</code>	Batch config (title, total, audience, weights, rules file paths)
<code>--total / --title / --audience</code>	Override the config
<code>--plan-only</code>	Print the distribution and stop
<code>--prompt-out P</code>	Write the full system prompt and all spec batches to P, then stop (subscription modes)
<code>--provider</code>	anthropic · kimi · openai · gemini · ollama · compatible · mock · file
<code>--input P</code>	With <code>--provider file</code> : the <code>{"items":[...]}</code> file the agent wrote
<code>--model / --effort / --temperature</code>	Model choice, reasoning depth (anthropic, kimi), sampling (not sent to Claude)
<code>--batch N / --max-calls N</code>	Items per request (default 5) / cap on requests (default 200)
<code>--dedup-db</code>	Also reject stems already in the target database
<code>--insert --env staging</code>	Write kept items to staging as inactive, pending teacher review
<code>--rules P</code>	Use a different rules file (default <code>scripts/generation-rules.json</code>)
<code>--out P</code>	Output JSON path (the <code>.md</code> sheet is written next to it)
<code>--no-json-mode</code>	Don't send <code>response_format</code> (compatible and kimi)